

Guía de balanceador de carga con HAProxy

Configuración, alta disponibilidad y buenas prácticas



Balanceador de carga con HAProxy en CentOS

Arquitectura

- Servidor Apache 1: Backend 1 (IP: 192.168.1.103).
- Servidor Apache 2: Backend 2 (IP: 192.168.1.104).
- Servidor Balanceador: Balanceador de carga (IP: 192.168.1.95).

Con esta disposición usaremos el balanceador de carga para equilibrar el rendimiento entre los dos servidores web y evitar sobrecargar alguno de ellos.

Configuración de los servidores web

En ambas maquinas instalaremos apache y permitiremos su trafico a traves del firewall

```
victor@asir:~$ sudo apt install apache2
Los paquetes indicados a continuación se instalaron de forma automática y ya
no son necesarios.
  libutempter0                                linux-modules-extra-6.14.0-15-generic
  linux-headers-6.14.0-15                     linux-tools-6.14.0-15
  linux-headers-6.14.0-15-generic             linux-tools-6.14.0-15-generic
  linux-modules-6.14.0-15-generic
Utilice «sudo apt autoremove» para eliminarlos.
```

```
Installing:
  apache2
```

```
victor@httpdserv:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; pres
   Active: active (running) since Wed 2026-01-21 12:50:36 UTC; 1min 27s ab
   Invocation: d7f6e9f93c134367bd6568b6ca4ee06f
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 12397 (apache2)
   Tasks: 55 (limit: 1873)
   Memory: 5.2M (peak: 5.8M)
   CPU: 42ms
   CGroup: /system.slice/apache2.service
           └─12397 /usr/sbin/apache2 -k start
             └─12399 /usr/sbin/apache2 -k start
               └─12400 /usr/sbin/apache2 -k start

ene 21 12:50:36 httpdserv systemd[1]: Starting apache2.service - The Apache HTTP Server
ene 21 12:50:36 httpdserv apache2[12397]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead. Please see the README file in /usr/share/httpd/errors for more details.
ene 21 12:50:36 httpdserv systemd[1]: Started apache2.service - The Apache HTTP Server
lines 1-17/17 (END)
```

```
victor@asir:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; pres
   Active: active (running) since Wed 2026-01-21 12:47:41 UTC; 1min 29s ab
   Invocation: 9b095cdd2b93482b8a10a80abfa2589f
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2216 (apache2)
   Tasks: 55 (limit: 1872)
   Memory: 5.2M (peak: 5.6M)
   CPU: 39ms
   CGroup: /system.slice/apache2.service
           └─2216 /usr/sbin/apache2 -k start
             └─2218 /usr/sbin/apache2 -k start
               └─2219 /usr/sbin/apache2 -k start

ene 21 12:47:41 asir systemd[1]: Starting apache2.service - The Apache HTTP Server
ene 21 12:47:41 asir apache2[2216]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead. Please see the README file in /usr/share/httpd/errors for more details.
ene 21 12:47:41 asir systemd[1]: Started apache2.service - The Apache HTTP Server
lines 1-17/17 (END)
```

```
victor@asir:~$ sudo ufw allow "Apache Full"
Rules updated
Rules updated (v6)
victor@asir:~$
```

```
victor@httpdserv:~$ sudo ufw status
Status: active

To Action From
--
Apache Full ALLOW Anywhere
22/tcp ALLOW Anywhere
Apache Full (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)

victor@httpdserv:~$
```

```
victor@asir:~$ sudo ufw status
Status: active

To Action From
--
Apache Full ALLOW Anywhere
22/tcp ALLOW Anywhere
Apache Full (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)

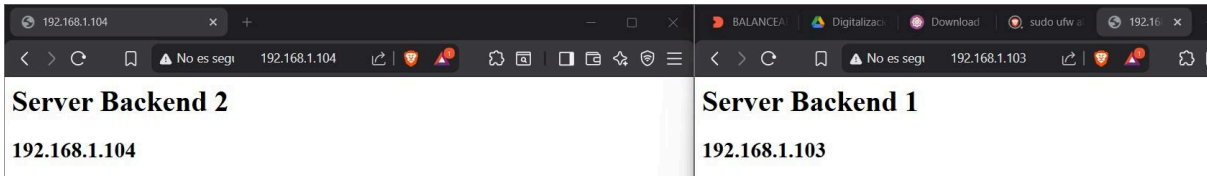
victor@asir:~$
```

Crearemos tambien una pequeña pagina de muestra en ambos para poder comprobar que nos estamos conectando a ellos y tambien para poder diferenciarlos

```
victor@httpdservr:~$ cat /var/www/html/index.html
index.html      old_index.html
victor@httpdservr:~$ cat /var/www/html/index.html
<h1>Server Backend 2</h1>
<h2>192.168.1.104</h2>
victor@httpdservr:~$

victor@asir:~$ cat /var/www/html/index.html
<h1>Server Backend 1</h1>
<h2>192.168.1.103</h2>
victor@asir:~$
```

Probamos a conectarnos y vemos que funciona



Configuración del servidor balanceador

Actualizamos nuestro sistema CentOS para tener todos los repositorios necesarios.

Una vez hecho esto instalamos haproxy

```
[victor@localhost ~]$ sudo dnf install haproxy -y
Última comprobación de caducidad de metadatos hecha hace 0:06:18, el mié 21 ene
2026 14:10:24.
Dependencias resueltas.
=====
Paquete      Arquitectura  Versión      Repositorio  Tam.
=====
Instalando:
haproxy      x86_64        2.8.14-3.el9  appstream    2.5 M
Resumen de la transacción
=====
Instalar 1 Paquete
```

Una vez instalado permitimos tráfico http en nuestro firewall

```
[victor@localhost ~]$ firewall-cmd --permanent --add-service=http
success
[victor@localhost ~]$ sudo firewall-cmd --reload
success
[victor@localhost ~]$
```

Configuración de HAproxy

Accedemos como root al archivo de configuración de HAproxy /etc/haproxy/haproxy.cfg y escribimos la siguiente configuración en el archivo:

```
GNU nano 5.6.1 /etc/haproxy/haproxy.cfg
global
    log /dev/log local0
    log /dev/log local1 notice
    maxconn 4000
    user haproxy
    group haproxy
    daemon
    stats socket /var/lib/haproxy/stats level admin

defaults
    log global
    mode http
    option httplog
    option dontlognull
    option redispatch
    timeout connect 10s
    timeout client 30s
    timeout server 30s
    timeout http-request 15s

frontend http_front
    bind *:80
    default_backend http_back
    option forwardfor
    http-request set-header X-Forwarded-Port %[dst_port]

backend http_back
    balance roundrobin
    server backend1 192.168.1.103:80 check maxconn 100
    server backend2 192.168.1.104:80 check maxconn 100
    option httpchk
    http-check send meth GET uri / ver HTTP/1.1 hdr Host localhost
    default-server inter 3s fall 3 rise 2

listen stats
    bind *:8080
    stats enable
```

Hecho esto reiniciamos el servicio de HAproxy y comprobamos su estado

```
[victor@localhost ~]$ sudo systemctl restart haproxy
[victor@localhost ~]$ sudo systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
   Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; pres>
   Active: active (running) since Fri 2026-01-23 09:22:40 CET; 3s ago
   Process: 87356 ExecStartPre=/usr/sbin/haproxy -f $CONFIG -f $CFGDIR -c >
   Main PID: 87358 (haproxy)
   Status: "Ready."
   Tasks: 3 (Limit: 10504)
   Memory: 7.1M (peak: 7.6M)
   CPU: 39ms
   CGroup: /system.slice/haproxy.service
           └─87358 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -f />
             └─87360 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -f />

ene 23 09:22:40 localhost.localdomain haproxy[87358]: [ALERT] (87358) : >
ene 23 09:22:40 localhost.localdomain haproxy[87358]: [NOTICE] (87358) : >
ene 23 09:22:40 localhost.localdomain haproxy[87358]: [NOTICE] (87358) : >
ene 23 09:22:40 localhost.localdomain systemd[1]: Started HAProxy Load Bala>
ene 23 09:22:40 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:41 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:41 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:42 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:42 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:42 localhost.localdomain haproxy[87360]: [WARNING] (87360) : >
ene 23 09:22:42 localhost.localdomain haproxy[87360]: [ALERT] (87360) : >
lines 1-23/23 (END)
```


Panel de estadísticas de HAproxy

Finalmente podremos acceder con la url `ip_balanceador:8080/stats` a un panel de estadísticas de nuestros backend y frontend

Statistics Report for HAPr

← → ↺ No seguro http://192.168.1.95:8080/stats

HAProxy version 2.8.14-c23fe91, released 2025/01/29

Statistics Report for pid 931 on localhost.localdomain

> General process information

pid = 931 (process #1, nbproc = 1, nbthread = 2)

uptime = 0d 0h03m29s; warnings = 0

system limits: memmax = unlimited; ulimit-n = 8032

maxsock = 8032; maxconn = 4000; reached = 0; maxpipes = 0

current conns = 1; current pipes = 0/0; conn rate = 0/sec; bit rate = 0.271 kbps

running tasks: 0/18; idle = 100 %

active UP

active UP, going down

active DOWN, going up

active or backup DOWN

active or backup DOWN for maintenance (MAINT)

active or backup SOFT STOPPED for maintenance

backup UP

backup UP, going down

backup DOWN, going up

not checked

active or backup DOWN for maintenance (MAINT)

active or backup SOFT STOPPED for maintenance

Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

Display option:

Scope :

Hide DOWN servers

Disable refresh

Refresh now

CSV export

JSON export (schema)

External resources:

Primary site

Updates v(2.8)

Online manual

http front

	Queue			Session rate			Sessions					Bytes		Denied		Errors			Warnings		Server											
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Downtme	Thrtle		
Frontend	0	2	-	0	2	-	0	2	4 000	6			12 339	6 825	0	0	3					OPEN										

http back

	Queue			Session rate			Sessions					Bytes		Denied		Errors			Warnings		Server									
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Downtme	Thrtle
backend1	0	0	-	0	4	-	0	1	100	12	12	30s	6 162	3 327	0	0	0	0	0	0	0	3m30s UP	L7OK/200 in 1ms	1/1	Y	-	0	0	0s	-
backend2	0	0	-	0	4	-	0	1	100	12	12	30s	6 177	3 498	0	0	0	0	0	0	0	3m30s UP	L7OK/200 in 1ms	1/1	Y	-	0	0	0s	-
Backend	0	0		0	7		0	1	400	24	24	30s	12 339	6 825	0	0						3m30s UP		2/2	2	0		0	0s	

stats

	Queue			Session rate			Sessions					Bytes		Denied		Errors			Warnings		Server										
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Downtme	Thrtle	
Frontend	0	1	-	0	1	-	0	1	4 000	1			1 123	49 725	0	0	0					OPEN									
Backend	0	0		0	0		0	0	400	0	0	0s	1 123	49 725	0	0						3m30s UP		0/0	0	0		0			

Si accedemos directamente a la url de nuestro servidor con el puerto 80 veremos que alternamos entre nuestro servidor apache 1 y 2, es decir, nuestro servidor balanceador seria el frontend, la pagina a la que accede el usuario, y los otros dos servidores servirian el mismo contenido web, de manera que se reparte la carga entre ambos servidores de manera equitativa a fin de no sobrecargar uno de ellos.